Grenzen verstehen. Struktur erkennen.

Lernskript mit Erklärungen, gerechneten Beispielen, 41 Kurzaufgaben, vier vollständigen Eigenaufträgen und einem 20-Punkte-Wochencheck.

05.–11. Oktober 2026 · Klausurvorbereitung für den 20. November · Stand 07.10.2026

So arbeitest du mit diesem Skript

1. Vorwissen aktivieren: Paritäts-DEA, Grammatikregeln, ε und ∅ ohne Vorlage erklären.

2. Pro Kapitel die Begriffe lesen und das gemeinsame Beispiel selbst nachzeichnen.

3. Den Eigenauftrag ohne Lösung bearbeiten. Die Lösungen stehen getrennt im Anhang.

4. Nach Fehlern: ersten falschen Schritt markieren, Regel erklären, Miniübung in der App lösen, später eine neue Variante bearbeiten.

Deine vier Etappen

Grenzen endlicher Automaten verstehen

Pumping ohne Quantorenfehler

Grammatiken, Syntaxbäume und Mehrdeutigkeit

Chomsky-Normalform vollständig herstellen

Wissen frei abrufen, selbst rechnen und begründen sind getrennte Ziele. Offene Beweise, Bäume und Grammatiken werden anhand von Kriterien selbst bewertet; diese Bewertung ist kein automatischer Kompetenznachweis.

DEIN ABLAUF

Acht Stunden mit Wiederholung

Einheit	Auftrag
Montag · 75 Min.	10 Min. Paritäts-DEA aus Woche 1 abrufen, 15 Min. Grenzen verstehen, 40 Min. Aufgabe 9 und 11b bearbeiten, 10 Min. korrigieren.
Dienstag · 75 Min.	10 Min. Speicherargument abrufen, 15 Min. Pumping-Beispiel, 40 Min. Aufgabe 11a und eine neue Variante, 10 Min. Fehlerliste.
Donnerstag · 75 Min.	10 Min. Grammatikregeln wiederholen, 15 Min. Baum lesen, 40 Min. Aufgabe 17 und eine eigene Variante, 10 Min. vergleichen.
Freitag · 75 Min.	10 Min. CNF-Regelformen abrufen, 15 Min. gemeinsames Beispiel, 40 Min. ε- und Kettenregeln entfernen, Terminale auslagern und lange Regeln zerlegen, 10 Min. prüfen.
Samstag · Block 1 · 75 Min.	10 Min. Abruf, 50 Min. zweite Grammatik vollständig umformen und Aufgabe 15 als Begriffsnetz beginnen, 15 Min. korrigieren. Danach zusätzliche Pause.
Samstag · Block 2 · 75 Min.	50 Min. ohne Lösungshilfe: Speicherargument, Pumping-Skizze, Syntaxbäume und zwei kleine CNF-Umformungen. 25 Min. anhand des Rasters selbst bewerten.
Sonntag · 30 Min.	10 Min. Regeln frei abrufen, 10 Min. Fehler und Merksätze prüfen, 10 Min. offene Aufgaben und Einstieg in CYK planen.

Mittwoch frei/Puffer; Pausen zusätzlich. Bei späterem Start die Reihenfolge verschieben, keine erledigte Einheit voraussetzen. Wiederhole nach etwa 1, 3, 7 und 14 Tagen innerhalb der Lernzeit.

KAPITEL 01

Grenzen endlicher Automaten verstehen

Ein Automat hat endlich viele Zustände, kann aber unendlich viele Wörter erkennen. Entscheidend ist, welche Erinnerung eine spätere Entscheidung braucht.

Deine Lernziele

Wissen: Endlicher Speicher und unterscheidbare Präfixe

Anwenden: Endlichen Speicher von unbeschränktem Vergleich unterscheiden und mit Restwörtern argumentieren.

Begründen: Warum ist „Die Sprache ist unendlich“ kein Beweis für Nichtregularität?

Begrenzt zählen und modulo zählen

Drei Nullen zählen braucht nur die Fälle 0, 1, 2, 3 und zu viele. Gerade/ungerade zählen braucht sogar nur zwei Fälle. Ein DEA kann also durchaus zählen, solange endlich viele Erinnerungen genügen.

Unbeschränkt vergleichen

In a^nb^n muss der Automat nach dem a-Block für beliebig großes n genau wissen, wie viele b noch fehlen. Zwei unterschiedliche a-Präfixe dürfen deshalb nicht dieselbe Erinnerung haben. Ein Keller kann dagegen pro a einen Marker ablegen.

Nicht regulär bedeutet nicht automatisch kontextfrei

a^nb^n ist mit $S \to aSb|\varepsilon$ kontextfrei. Für andere nicht reguläre Sprachen reicht ein Keller nicht. Eine Nichtregularitätsbegründung allein bestimmt daher keine genaue höhere Sprachklasse.

Korrektur zu Aufgabe 11b

Die Musterlösung erklärt L2 fälschlich für nicht regulär. Tatsächlich genügt ein DEA mit vier Paritätszuständen für a und b; c ändert nichts. Start (gerade,gerade), einzig akzeptierend (gerade,ungerade). ε wird abgelehnt.

Fundstellen: TGI_02_02_RegulaereSprachen.pdf · PDF-S. 70–81; Aufgaben (1).pdf · S. 9, 11

Einmal vollständig durchrechnen

Vergleiche L1={w∈{a,b}*: Anzahl a gerade} und L2={$a^n b^n$:n≥0}. Warum ist nur die erste Sprache regulär?

1. L1: Zwei Zustände G/U reichen. Start und einziger Endzustand G; a wechselt G↔U, b bleibt im Zustand.

2. L2: Nach a^i und a^j mit i≠j unterscheidet das Restwort b^i: $a^i b^i$ gehört dazu, $a^j b^i$ nicht.

3. Wären beide Präfixe im selben DEA-Zustand, müsste dieselbe Fortsetzung gleich entschieden werden. Unendlich viele paarweise unterscheidbare Präfixe brauchen unendlich viele Zustände.

Zustand	Übergänge / Annahme
G	a→U, b→G; Start und akzeptierend
U	a→G, b→U; nicht akzeptierend

Das Verfahren merken

1. Prüfe, welche Information die bisherige Eingabe hinterlassen muss.

2. Frage, ob endlich viele Fälle (z. B. modulo 2) genügen.

3. Für Nichtregularität: finde beliebig viele Präfixe, die unterschiedliche Fortsetzungen verlangen.

Typischer Fehler: Unendlichkeit der Sprache und unbeschränkter notwendiger Speicher sind verschiedene Dinge.

Jetzt bist du dran

A · Wissen frei abrufen

Endlicher Speicher und unterscheidbare Präfixe: Schreibe die Regeln und Voraussetzungen aus dem Gedächtnis auf.

B · Vollständiger Eigenauftrag

Über {a,b,c}: L={w: Anzahl a gerade und Anzahl b ungerade}. Konstruiere einen vollständigen DEA mit Zustandsbedeutungen und prüfe ε, b, aabc.

Notiere Zwischenstände, nicht nur das Endergebnis. Zeichnungen auf Papier; Lösung im Anhang. Bei Bedarf in der App erst mit Starthilfe, dann mit weniger Hilfe und schließlich allein rechnen.

C · Begründen

Warum ist „Die Sprache ist unendlich“ kein Beweis für Nichtregularität?

Deine Fehlernotiz

Mein erster unsicherer Schritt: _____
Die richtige Regel in meinen Worten: _____
Nächste Variante und Wiederholungstermin: _____

Üben und entscheiden · 1–5

1. Warum kann a* trotz unendlich vieler Wörter regulär sein?

A) Ein akzeptierender Zustand mit a-Schleife reicht.

B) Jedes Wort braucht einen eigenen Zustand.

C) Nur endliche Sprachen sind regulär.

2. Welche Erinnerung fehlt einem DEA für $a^n b^n$ mit beliebigem n?

A) Ob überhaupt ein a vorkam.

B) Die unbeschränkt große exakte Zahl der a.

C) Ob die letzte Stelle b war.

3. Eine Sprache ist nicht regulär. Was folgt sicher?

A) Sie ist kontextfrei.

B) Sie ist unentscheidbar.

C) Es gibt keinen endlichen Automaten, der sie erkennt.

4. $L=\{a^n b^n : 0 \leq n \leq 100\}$. Welche Aussage stimmt?

A) Nicht regulär wegen gleicher Zahlen.

B) Regulär, denn es sind endlich viele Wörter.

C) Nicht entscheidbar.

5. Für $L=\{a^n b^n : n \geq 0\}$: Welches Restwort trennt die Präfixe aa und aaa?

A) bb

B) ε

C) aaaa

Üben und entscheiden · 6–10

6. Wie viele Kombinationen gibt es für (Anzahl a modulo 2, Anzahl b modulo 3)?

Deine Antwort: _____

7. Paritäts-DEA: GG,GU,UG,UU; erste Komponente zählt a, zweite b; G=gerade, U=ungerade. Start GG. a kippt die erste, b die zweite, c bleibt. Zustand nach abac?

Deine Antwort: _____

8. Paritäts-DEA mit Q={GG,GU,UG,UU}. Akzeptiert werden gerade viele a und ungerade viele b. Gib F an.

Deine Antwort: _____

9. Sprache über {a,b,c}: a-Anzahl gerade, b-Anzahl ungerade. Gehört ε dazu?

A) Ja, null ist gerade.

B) Nein, null b ist nicht ungerade.

10. Warum hilft b^i beim Vergleich von a^i und a^j ($i \neq j$) für $a^n b^n$?

A) Es akzeptiert beide Präfixe.

B) Nur $a^i b^i$ hat gleich lange Blöcke.

C) Es verändert die Anzahl a.

Pumping ohne Quantorenfehler

Eine genügend lange Fahrt durch einen endlichen Automaten wiederholt einen Zustand. Die Schleife kann öfter oder gar nicht durchlaufen werden. Das ist die Pumping-Idee.

Deine Lernziele

Wissen: Pumping-Lemma samt Quantoren

Anwenden: Einen kurzen Pumping-Widerspruch mit korrekter Wortwahl und allen erlaubten Zerlegungen schreiben.

Begründen: Warum reicht eine selbst gewählte ungünstige Zerlegung nicht als Pumping-Beweis?

Die drei Bedingungen

Für reguläres L gibt es $p \geq 1$. Jedes Wort $w \in L$ mit $|w| \geq p$ hat mindestens eine Zerlegung $w = xyz$ mit $|xy| \leq p$, $|y| \geq 1$ und $xy^iz \in L$ für alle $i \geq 0$. y ist der wiederholbare, nichtleere Schleifenteil.

Die Reihenfolge ist entscheidend

Erst wird p vorgegeben, dann w gewählt. Danach musst du jede zulässige Zerlegung abdecken. Der Pumpfaktor darf von der Zerlegung abhängen. i=0 lässt y weg, i=2 verdoppelt y; x und z bleiben gleich.

Ein Widerspruch ist kein Regularitätsbeweis

Wenn du keine Widerlegung findest, ist die Sprache noch nicht regulär bewiesen. Ein DEA oder ein RegEx ist ein positiver Nachweis. Auch eine einzelne schlechte Zerlegung beweist keine Nichtregularität.

Aufgabe 11a übertragen

Für Anzahl a plus Anzahl b gleich Anzahl c wähle a^pc^p. Jede erlaubte Pumpzone liegt nur in a. Abpumpen ändert die linke Anzahl, aber nicht die Zahl der c. So scheitert jede erlaubte Zerlegung.

Fundstellen: TGI_02_02_RegulaereSprachen.pdf · PDF-S. 72–81; Aufgaben (1).pdf · S. 11

Einmal vollständig durchrechnen

Zeige: $L=\{a^n b^n : n \geq 0\}$ ist nicht regulär.

1. Angenommen L ist regulär. Sei $p \geq 1$ eine Pumping-Länge; $w=a^p b^p$ liegt in L und hat Länge $2p \geq p$.

2. Für jede erlaubte Zerlegung liegt y vollständig im ersten a-Block: $y=a^k$ mit $1 \leq k \leq p$.

3. Mit i=0 entsteht a^(p−k)b^p. Es hat weniger a als b und liegt nicht in L.

4. Jede erlaubte Zerlegung scheitert an i=0. Das widerspricht dem Pumping-Lemma; L ist nicht regulär.

Teil	Erklärung
x	Präfix nur aus a, eventuell ε
$y=a^k$	$1 \leq k \leq p$, nichtleer, ganz im ersten Block
z	Übrige a und alle p b
xy^0z	a^(p−k)b^p, ungleiche Anzahlen

Das Verfahren merken

1. Nimm Regularität an und lasse p eine beliebige garantierte Pumping-Länge sein.

2. Wähle $w \in L$ mit $|w| \geq p$ in Abhängigkeit von p.

3. Betrachte jede Zerlegung w=xyz mit $|xy| \leq p$ und $|y| \geq 1$.

4. Wähle einen Pumpfaktor $i \geq 0$ und zeige $xy^iz \notin L$.

Typischer Fehler: Wähle nicht einfach y selbst. Formuliere $y=a^k$ für alle durch die Bedingungen möglichen k.

Jetzt bist du dran

A · Wissen frei abrufen

Pumping-Lemma samt Quantoren: Schreibe die Regeln und Voraussetzungen aus dem Gedächtnis auf.

B · Vollständiger Eigenauftrag

Zeige: $L=\{a^n b^{2n} : n \geq 0\}$ ist nicht regulär. Gib Wortwahl, beliebige Zerlegung, Pumpfaktor und Widerspruch an.

Notiere Zwischenstände, nicht nur das Endergebnis. Zeichnungen auf Papier; Lösung im Anhang. Bei Bedarf in der App erst mit Starthilfe, dann mit weniger Hilfe und schließlich allein rechnen.

C · Begründen

Warum reicht eine selbst gewählte ungünstige Zerlegung nicht als Pumping-Beweis?

Deine Fehlernotiz

Mein erster unsicherer Schritt: _____
Die richtige Regel in meinen Worten: _____
Nächste Variante und Wiederholungstermin: _____

Üben und entscheiden · 1–5

1. Für $L=\{a^n b^n : n \geq 0\}$ und vorgegebene Pumping-Länge p: welches Wort ist für den Standardbeweis geeignet?

A) ab unabhängig von p

B) $a^p b^p$

C) $a^p b^{(p+1)}$

2. $w=a^p b^p$, $|xy| \leq p$, $|y| \geq 1$. Welche Form hat y in jeder erlaubten Zerlegung?

A) a^k mit $1 \leq k \leq p$

B) ε

C) $a^k b^k$

3. Wie viele erlaubte Zerlegungen musst du im Widerspruch abdecken?

A) Eine besonders schlechte.

B) Alle erlaubten Zerlegungen.

C) Nur x=ε.

4. Du findest keinen Pumping-Widerspruch. Was folgt?

A) Die Sprache ist regulär.

B) Die Sprache ist kontextfrei.

C) Noch keine positive Klassifikation.

5. x=aa, y=a, z=bbb. Gib $xy^o z$ als Wort ein.

Deine Antwort: _____

Üben und entscheiden · 6–10

6. x=a, y=aa, z=bbb. Gib xy^2z als Wort ein.

Deine Antwort: _____

7. Für a^pb^{2p} mit p=5 und $y=a^2$: Wie viele b wären nach Abpumpen erforderlich, damit das Wort wieder die Form a^nb^{2n} hätte?

Deine Antwort: _____

8. Welche Pumpfaktoren muss die gute Zerlegung einer regulären Sprache vertragen?

A) Nur positive gerade i.

B) Nur i=1.

C) Alle ganzen i≥0.

9. L={w über {a,b,c}: |w|a+|w|b=|w|c}. Ist a^pc^p als Zeugenwort zulässig?

A) Ja; p+0=p und Länge 2p.

B) Nein, es muss ein b vorkommen.

10. Ein Beweis setzt y=ε und pumpt ab. Was ist der unmittelbare Fehler?

A) y darf nicht leer sein.

B) Abpumpen ist nie erlaubt.

C) p muss gleich 1 sein.

Grammatiken, Syntaxbäume und Mehrdeutigkeit

Eine kontextfreie Grammatik beschreibt, wie größere Ausdrücke aus kleineren entstehen. Ein Syntaxbaum zeigt die Struktur, die in einer bloßen Zeichenfolge verborgen bleibt.

Deine Lernziele

Wissen: Kontextfreie Grammatik, Baum und Mehrdeutigkeit

Anwenden: Eine Ableitung als Baum lesen und Mehrdeutigkeit mit zwei Bäumen desselben Wortes nachweisen.

Begründen: Macht die Regel „Punkt vor Strich“ diese Grammatik eindeutig?

Die vier Bestandteile

G=(Σ,V,S,R): Terminalalphabet Σ, davon getrennte Variablen V, Startvariable S∈V und endliche Regelmenge R. Kontextfrei heißt: links steht genau eine Variable. Rechts darf ein beliebiges Wort aus Terminalen und Variablen einschließlich ε stehen.

Ableitung und Baum

A→aB ist eine Regel; S⇒aB⇒ab ist eine Ableitungsfolge. Innere Baumknoten sind Variablen, ihre Kinder bilden genau die rechte Seite einer angewendeten Regel. Die Blätter von links nach rechts bilden das Wort.

Mehrdeutigkeit nachweisen

Eine Grammatik ist mehrdeutig, wenn es mindestens ein Wort mit zwei verschiedenen Syntaxbäumen gibt. Zwei unterschiedliche Linksableitungen desselben Wortes sind ebenfalls ein Nachweis. Unterschiedliche Reihenfolge unabhängiger Ersetzungen genügt nicht.

Warum Parser das brauchen

Ein Parser rekonstruiert die Struktur eines Programms oder Ausdrucks. Ohne eindeutige Struktur kann a+a*a verschieden gruppiert werden. Mit E→E+T|T, T→T*F|F, F→a erzwingt die Grammatik Multiplikation vor Addition und Linksassoziativität.

Fundstellen: TGI_03_KontextfreieSprachen.pdf · PDF-S. 7–15; Aufgaben (1).pdf · S. 15, 17; Altklausur.pdf · S. 10

Einmal vollständig durchrechnen

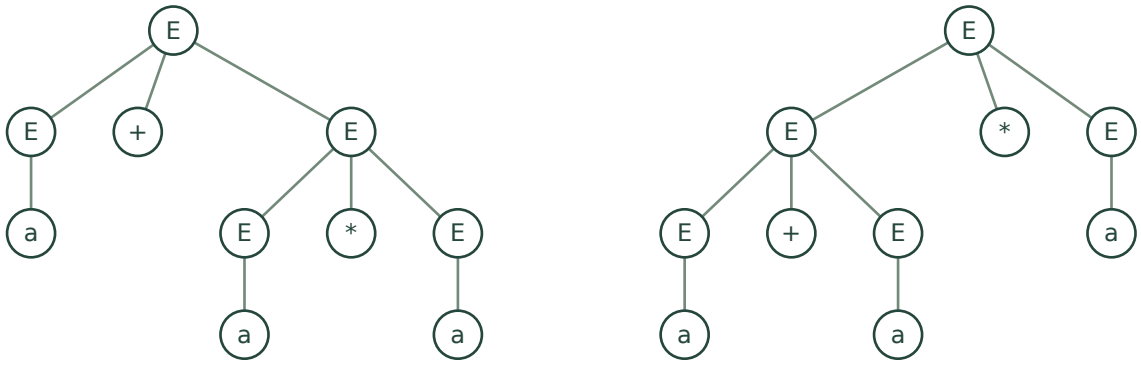
Grammatik: Σ={a,+,*}, V={E}, Start E; E→E+E | E*E | a. Zeige Mehrdeutigkeit für a+a*a.

1. Baum 1: Wurzel E→E+E; linkes E→a; rechtes E→E*E mit beiden Kindern E→a. Struktur a+(a*a).

2. Linksableitung 1: E⇒E+E⇒a+E⇒a+E*E⇒a+a*E⇒a+a*a.

3. Baum 2: Wurzel E→E*E; links E→E+E mit a-Blättern; rechts E→a. Struktur (a+a)*a.

4. Linksableitung 2: E⇒E*E⇒E+E*E⇒a+E*E⇒a+a*E⇒a+a*a. Beide haben dasselbe Blattwort, aber verschiedene Wurzeln.



Beide Blattwörter: a+a*a. Links Addition an der Wurzel, rechts Multiplikation.

Das Verfahren merken

1. Wurzel ist das Startsymbol; jede Verzweigung muss einer Grammatikregel entsprechen.

2. Ersetze Variablen, bis alle Blätter Terminale (oder ε) sind.

3. Lies Blätter von links nach rechts: Sie müssen exakt das geforderte Wort ergeben.

4. Suche für dasselbe Wort eine andere Baumstruktur, nicht nur eine andere Reihenfolge unabhängiger Schritte.

Typischer Fehler: Klammern in unseren Strukturangaben erklären den Baum; sie gehören nicht zum Terminalwort a+a*a.

Jetzt bist du dran

A · Wissen frei abrufen

Kontextfreie Grammatik, Baum und Mehrdeutigkeit: Schreibe die Regeln und Voraussetzungen aus dem Gedächtnis auf.

B · Vollständiger Eigenauftrag

Σ={a,+,*}, V={E}, Start E; E→E+E|E*E|a. Zeichne zwei Bäume für a*a+a und gib zwei Linksableitungen an.

Notiere Zwischenstände, nicht nur das Endergebnis. Zeichnungen auf Papier; Lösung im Anhang. Bei Bedarf in der App erst mit Starthilfe, dann mit weniger Hilfe und schließlich allein rechnen.

C · Begründen

Macht die Regel „Punkt vor Strich“ diese Grammatik eindeutig?

Deine Fehlernotiz

Mein erster unsicherer Schritt: _____
Die richtige Regel in meinen Worten: _____
Nächste Variante und Wiederholungstermin: _____

Üben und entscheiden · 1–5

1. Welche Regel ist kontextfrei, wenn A,B Variablen und a,b Terminale sind?

A) AB→ab

B) aA→Ab

C) A→aBB

2. Was liest du an den Blättern eines vollständigen Syntaxbaums von links nach rechts?

A) Die verwendeten Variablennamen.

B) Das abgeleitete Terminalwort (ε trägt kein Zeichen bei).

C) Die Reihenfolge aller Rechenschritte.

3. Was reicht als Nachweis der Mehrdeutigkeit einer Grammatik?

A) Zwei Bäume zu zwei unterschiedlichen Wörtern.

B) Zwei verschiedene Bäume für dasselbe Wort.

C) Zwei beliebige Ableitungen, auch mit identischem Baum.

4. S→AB, A→a, B→b. Erst A oder erst B ersetzen: beweist das Mehrdeutigkeit?

A) Ja, jede andere Reihenfolge genügt.

B) Nein, beide Reihenfolgen ergeben denselben Baum.

5. S→aSb|ε. Ein Baum verwendet zweimal S→aSb und dann S→ε. Welches Terminalwort liest du?

Deine Antwort: _____

Üben und entscheiden · 6–10

6. S→aSb|ab. Nach dreimaliger Anwendung von S→aSb folgt S→ab. Wie lang ist das fertige Wort?

Deine Antwort: _____

7. E→E+E|E*E|a. Welche Wurzelregel gehört zur Struktur (a+a)*a?

A) E→E+E

B) E→E*E

C) E→a

8. E→E+T|T; T→T*F|F; F→a. Welche Struktur hat a+a*a?

A) (a+a)*a

B) a+(a*a)

9. Eine Grammatik ist mehrdeutig. Was folgt für ihre Sprache?

A) Jede Grammatik dieser Sprache ist mehrdeutig.

B) Die Sprache ist nicht regulär.

C) Nur diese Grammatik ist damit als mehrdeutig nachgewiesen.

10. E→E+E|E*E|a. In der Linksableitung steht a+E*E. Ersetze genau die linkeste Variable durch a.

Deine Antwort: _____

Chomsky-Normalform vollständig herstellen

CNF vereinheitlicht Grammatikregeln. So kannst du Wörter später mit CYK systematisch prüfen. Die Umformung verändert die Struktur der Grammatik, aber nicht ihre Sprache.

Deine Lernziele

Wissen: CNF-Regelformen und Umformungsphasen

Anwenden: Eine kontextfreie Grammatik samt ε-Fall in strenge CNF umformen und die Sprache erhalten.

Begründen: Nach S0→S|ε sind alle alten Regeln in CNF. Ist die Gesamtgrammatik bereits in strenger CNF?

Erlaubte Regeln

A→BC: genau zwei Variablen. A→a: genau ein Terminal. Zusätzlich S0→ε, wenn ε zur Sprache gehört und S0 niemals rechts steht. Nicht erlaubt sind A→B, A→aB oder A→BCD. CNF sagt nichts über Eindeutigkeit aus.

Nullable Variablen finden

Eine Variable heißt nullable, wenn sie in mehreren Schritten ε erzeugen kann. Beginne mit direkten ε-Regeln und ergänze Variablen, deren gesamte rechte Seite bereits nullable ist. Wiederhole bis keine neue Variable entsteht.

ε-Regeln ohne Sprachverlust entfernen

Für jedes Vorkommen einer nullable Variable ergänze auch die rechte Seite ohne dieses Vorkommen. Bei mehreren Vorkommen alle Kombinationen beachten. Entferne alte ε-Regeln, aber erhalte ε nötigenfalls am neuen Start.

Kettenregeln und Zyklen

Für A→B übernimm alle Nicht-Kettenregeln aller per Kette erreichbaren Variablen. Bei Zyklen jede Variable nur einmal neu besuchen. Erst danach Kettenregeln löschen; bloßes Löschen verliert Wörter.

Terminale auslagern und binarisieren

In gemischten oder langen Regeln ersetze a durch eine neue Variable Ta mit Ta→a. A→BCD wird A→BX und X→CD. Einzelne Terminalregeln A→a bleiben. Neue Namen müssen frisch und konsistent sein.

Quellenhinweis und Schlusskontrolle

PDF-S. 33 endet mit S′→S|ε. Für strenge CNF muss S′→S noch entfernt werden, indem S′ die Nicht-Kettenregeln von S übernimmt. Prüfe ε, kurze Wörter und jede Regelform. Worttests finden Fehler; sie beweisen allein keine Sprachgleichheit.

Fundstellen: TGI_03_KontextfreieSprachen.pdf · PDF-S. 17–33

Einmal vollständig durchrechnen

Σ={a,b}, V={S}, Start S; S→aSb | ε. Überführe die Grammatik in strenge CNF.

1. Neuer Start S0→S|ε. S ist nullable; das ε der Sprache bleibt nur an S0 erhalten.

2. S→aSb|ab: Das innere S darf entfallen. S→ε wird entfernt. S0→S|ε bleibt zunächst.

3. Kettenregel entfernen: S0→aSb|ab|ε, S→aSb|ab.

4. A→a, B→b einführen: S0→ASB|AB|ε und S→ASB|AB.

5. Mit C→SB: S0→AC|AB|ε; S→AC|AB; C→SB; A→a; B→b. Start S0, Variablen {S0,S,C,A,B}.

6. Jede Regel ist zulässig; S0 steht nie rechts. AC erzeugt ein zusätzliches a/b-Paar, AB das letzte Paar; S0→ε erhält n=0. Sprache bleibt $\{a^n b^n : n \geq 0\}$.

Variable	Rechte Seiten in CNF
S0	AC | AB | ε
S	AC | AB
C	SB
A	a
B	b

Start S0; Σ={a,b}, V={S0,S,C,A,B}. S0 kommt niemals rechts vor.

Das Verfahren merken

1. Sichere ε mit neuem Start S0, das niemals rechts steht.

2. Bestimme alle nullable Variablen (die ε erzeugen können); ergänze Auslassungsvarianten und entferne alte ε-Regeln.

3. Entferne Kettenregeln A→B durch Übernahme aller erreichbaren Nicht-Kettenregeln.

4. Ersetze Terminale in längeren rechten Seiten durch Terminalvariablen.

5. Zerlege rechte Seiten mit mehr als zwei Variablen; prüfe jede Regel und die Spracherhaltung.

Typischer Fehler: ε darf nicht verloren gehen. Das neue Startsymbol darf nicht in rekursiven rechten Seiten erscheinen.

Jetzt bist du dran

A · Wissen frei abrufen

CNF-Regelformen und Umformungsphasen: Schreibe die Regeln und Voraussetzungen aus dem Gedächtnis auf.

B · Vollständiger Eigenauftrag

Σ={a,b}, V={S}, Start S; S→aSbb|ε. Forme vollständig in strenge CNF um und begründe die Spracherhaltung auch für ε.

Notiere Zwischenstände, nicht nur das Endergebnis. Zeichnungen auf Papier; Lösung im Anhang. Bei Bedarf in der App erst mit Starthilfe, dann mit weniger Hilfe und schließlich allein rechnen.

C · Begründen

Nach S0→S|ε sind alle alten Regeln in CNF. Ist die Gesamtgrammatik bereits in strenger CNF?

Deine Fehlernotiz

Mein erster unsicherer Schritt: _____
Die richtige Regel in meinen Worten: _____
Nächste Variante und Wiederholungstermin: _____

Üben und entscheiden · 1–6

1. Variablen A,B,C, Terminal a: Welche Regel ist in strenger CNF?

A) A→aB

B) A→BC

C) A→B

2. Wann ist S0→ε in der verwendeten CNF-Konvention erlaubt?

A) Bei jeder Variablen.

B) Wenn S0 Start ist, nie rechts vorkommt und ε erhalten werden muss.

C) Nur wenn die gesamte Sprache leer ist.

3. Start S, Variablen S,A,B,C; S→AB|c, A→ε, B→C, C→ε. Welche Variablen sind nullable?

Deine Antwort: _____

4. In S→ABA sind A und B nullable. Welche verschiedenen NICHTLEEREN rechten Seiten entstehen durch Behalten oder Auslassen der Vorkommen (Original einschließen)?

Deine Antwort: _____

5. A→B, B→C|b, C→a. Welche Nicht-Kettenregeln übernimmt A nach vollständiger Kettenelimination?

A) A→a

B) A→b

C) A→a|b

6. A→B|a, B→A|b. Was erhalten A und B nach Kettenelimination?

A) Beide →a|b

B) Nur A→a und B→b

C) Keine Regeln

Üben und entscheiden · 7–11

7. Die rechte Seite A→BCDE enthält nur Variablen. Wie viele frische Hilfsvariablen brauchst du bei der üblichen schrittweisen Binarisierung dieser einzelnen Regel?

Deine Antwort: _____

8. A→aBC wird mit T→a und X→BC ersetzt. Welche passende Regel erhält A?

A) A→TX

B) A→aX

C) A→TBC

9. Alle alten Regeln sind CNF, aber neu steht S0→S|ε. Was fehlt noch?

A) Nichts.

B) S0→S als Kettenregel entfernen und passende Regeln übernehmen.

C) ε in jedes Nichtterminal kopieren.

10. Was garantiert eine korrekte CNF-Umformung?

A) Dieselbe Sprache, aber nicht unbedingt dieselben Bäume.

B) Eindeutigkeit.

C) Regulärität.

11. Alte und neue Grammatik erzeugen bis Länge 5 dieselben Wörter. Ist Sprachgleichheit damit bewiesen?

A) Ja.

B) Nein, längere Wörter könnten abweichen.

WOCHENCHECK · 20 PUNKTE

Was bleibt ohne Vorlage?

50 Minuten bearbeiten, anschließend 25 Minuten auswerten. 4 Punkte automatisch prüfbare Ergebnisse, 16 Punkte freie Lösungen mit Selbstbewertung. Eigene Trainingsmarke: 16/20 und beide CNF-Aufgaben bearbeiten. Kein offizieller Notenschlüssel.

1. Welche Sprache ist sicher regulär? (1 P.)

A) $\{a^n b^n : n \geq 0\}$

B) Wörter über {a,b} mit a-Anzahl durch 3 teilbar

C) Jede unendliche Sprache

2. $w=a^6b^6$, Zerlegung mit y=aa. Wie viele a bleiben beim Pumpfaktor i=0? (1 P.)

3. Grammatik mit Start S: S→AB|a, A→B, B→ε, C→c. Gib alle nullable Variablen an. (1 P.)

4. Variablen S,A,B,C, Terminal a: Welche Regel ist in strenger CNF? (1 P.)

A) A→aB

B) A→B

C) A→BC

5. Vergleiche $L1=\{a^n b^n : 0 \leq n \leq 7\}$ und $L2=\{a^n b^n : n \geq 0\}$. Welche ist regulär? Begründe beide Aussagen. (1 P.)

Ausführlicher Lösungsweg auf einem eigenen Blatt.

6. Zeige mit Pumping: $L=\{0^n 1^{3n} : n \geq 1\}$ ist nicht regulär. Erfasse alle erlaubten Zerlegungen. (2 P.)

Ausführlicher Lösungsweg auf einem eigenen Blatt.

7. Σ={a,-,*}, V={E}, Start E; E→E-E|E*E|a. Zeichne zwei Syntaxbäume für a-a*a und begründe Mehrdeutigkeit. Klammern dürfen nur die Struktur erläutern. (3 P.)

Ausführlicher Lösungsweg auf einem eigenen Blatt.

8. Σ={a,b}, V={S,A}, Start S; S→A|b, A→aAb|ε. Forme vollständig in strenge CNF um und begründe die erhaltene Sprache. (5 P.)

Ausführlicher Lösungsweg auf einem eigenen Blatt.

9. Σ={a,b}, V={S}, Start S; S→aSbb|abb. Stelle strenge CNF her. Erkläre ausdrücklich, ob ε dazugehören darf. (5 P.)

Ausführlicher Lösungsweg auf einem eigenen Blatt.

Erst nach dem eigenen Versuch

Gleichwertige fachlich korrekte Lösungen zählen. Nach dem Aufdecken ist eine sofortige Wiederholung Lernen mit Unterstützung, kein neuer unabhängiger Erfolg. Schreibe einen Merksatz und übe an einem späteren Tag eine andere Variante.

Grenzen endlicher Automaten verstehen

Abrufwissen: Ein DEA besitzt endlich viele Erinnerungsfälle. Unendlich viele Wörter sind mit Schleifen möglich; modulo-Zählen ist regulär. Zwei Präfixe sind unterscheidbar, wenn eine gemeinsame Fortsetzung nur eines davon in die Sprache bringt.

Eigenauftrag

1. Vier Zustände GG, GU, UG, UU; erste Komponente Parität a, zweite b. Start GG, F={GU}.

2. a kippt die erste, b die zweite Komponente; c lässt beide gleich. GG:(UG,GU,GG); GU:(UU,GG,GU); UG:(GG,UU,UG); UU:(GU,UG,UU) für (a,b,c).

3. ε abgelehnt; b und aabc angenommen. Endlich viele Restfälle reichen trotz unbegrenzt langer Wörter.

Begründung: a* ist unendlich und wird von einem einzigen akzeptierenden Zustand mit a-Schleife erkannt. Entscheidend ist die Zahl unterscheidbarer Erinnerungsfälle, nicht die Zahl der Wörter.

1. Ein akzeptierender Zustand mit a-Schleife reicht. — Die Schleife erzeugt beliebig viele Wiederholungen, ohne zusätzliche Zustände zu brauchen.

2. Die unbeschränkt große exakte Zahl der a. — Nur „a gesehen“ oder „letztes b“ genügt nicht für den exakten späteren Längenvergleich.

3. Es gibt keinen endlichen Automaten, der sie erkennt. — Nichtregularität schließt endliche Automaten aus, bestimmt aber keine einzelne höhere Klasse.

4. Regulär, denn es sind endlich viele Wörter. — Die feste Grenze macht die Sprache endlich. Ein RegEx kann alle 101 Wörter aufzählen.

5. bb — aabb gehört zu L, aaabb nicht. Dasselbe Restwort führt zu verschiedenen Entscheidungen.

6. 6 — Zwei mögliche erste und drei mögliche zweite Komponenten: 2·3=6.

7. GU — GG→UG→UU→GU→GU. Zwei a sind gerade, ein b ungerade.

8. {GU} — Nur GU erfüllt beide Bedingungen; GG genügt wegen gerader b-Anzahl nicht.

9. Nein, null b ist nicht ungerade. — Beide Bedingungen müssen gelten. ε erfüllt die a-Bedingung, verletzt aber die b-Bedingung.

10. Nur $a^i b^i$ hat gleich lange Blöcke. — Ein gemeinsames Restwort unterscheidet die beiden Präfixe. Im selben DEA-Zustand wäre das unmöglich.

Pumping ohne Quantorenfehler

Abrufwissen: Regulär ⇒ es gibt $p \geq 1$, für alle $w \in L$ mit $|w| \geq p$ gibt es $w = xyz$ mit $|xy| \leq p$, $|y| \geq 1$ und für alle $i \geq 0$: $xy^i z \in L$. Ein Widerspruch muss jede erlaubte Zerlegung treffen.

Eigenauftrag

1. Regularität annehmen, Pumping-Länge $p \geq 1$, $w = a^p b^{2p} \in L$ mit $|w| = 3p \geq p$.

2. Für jede erlaubte Zerlegung gilt $y = a^k$, $1 \leq k \leq p$.

3. i=0 ergibt a^(p−k)b²ᵖ. Erforderlich wären 2(p−k) b, tatsächlich sind es 2p.

4. Da $k \geq 1$, ist $2(p-k) \neq 2p$; kein erlaubtes y rettet die Eigenschaft. Widerspruch.

Begründung: Bei einer regulären Sprache muss nur eine geeignete Zerlegung existieren, die alle Pumpfaktoren verträgt. Eine schlechte Zerlegung widerlegt diese Existenz nicht. Der Widerspruch muss jede erlaubte Zerlegung ausschließen.

1. $a^p b^p$ — $a^p b^p$ liegt in L und hat Länge mindestens p. ab kann zu kurz sein; die dritte Wahl liegt nicht in L.

2. a^k mit $1 \leq k \leq p$ — Die gesamte Pumpzone liegt in den ersten p Zeichen, also ausschließlich im a-Block, und ist nicht leer.

3. Alle erlaubten Zerlegungen. — Das Lemma verspricht mindestens eine gute Zerlegung. Du musst ausschließen, dass irgendeine davon funktioniert.

4. Noch keine positive Klassifikation. — Ein gescheiterter Versuch ist kein Regularitätsbeweis. Konstruiere etwa einen DEA für einen positiven Nachweis.

5. aabbb — $y^0 = \varepsilon$. x und z bleiben: aa·bbb=aabbb.

6. aaaaabbb — Ein a aus x plus zweimal zwei a aus y ergibt fünf a, danach drei b.

7. 6 — Nach Abpumpen bleiben 3 a. Passend wären 2·3=6 b; tatsächlich bleiben 10 b.

8. Alle ganzen $i \geq 0$. — Auch i=0 (Entfernen) gehört dazu. i=1 ist lediglich das unveränderte Ausgangswort.

9. Ja; p+0=p und Länge 2p. — Die Definition fordert keine positive Anzahl b. Bei i=0 sinkt nur die a-Anzahl, die c-Anzahl bleibt.

10. y darf nicht leer sein. — $|y| \geq 1$ ist Voraussetzung. Ohne nichtleeres y verändert Pumpen das Wort nicht.

Grammatiken, Syntaxbäume und Mehrdeutigkeit

Abrufwissen: Links einer kontextfreien Regel steht genau eine Variable. Wurzel=Startsymbol, Verzweigungen=Regeln, Blattfolge=Wort. Mehrdeutigkeit bedeutet zwei verschiedene Bäume für mindestens ein Wort.

Eigenauftrag

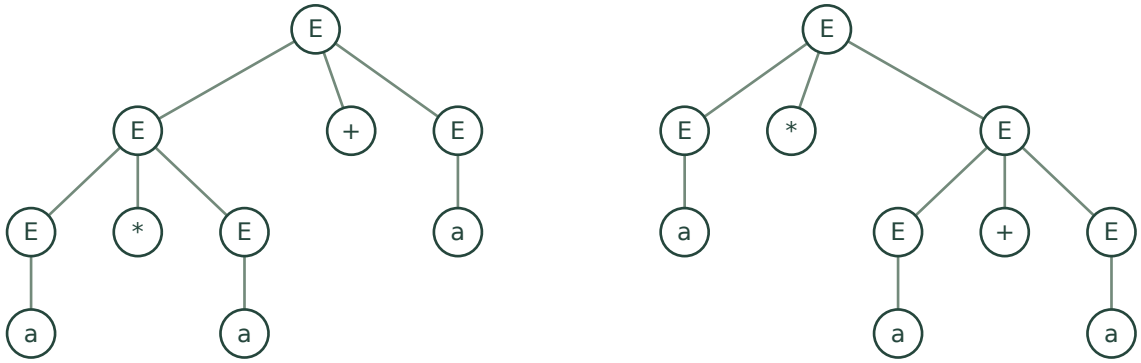
1. Baum 1: Addition an der Wurzel, links Multiplikation. Struktur (a*a)+a.

2. E⇒E+E⇒E*E+E⇒a*E+E⇒a*a+E⇒a*a+a.

3. Baum 2: Multiplikation an der Wurzel, rechts Addition. Struktur a*(a+a).

4. E⇒E*E⇒a*E⇒a*E+E⇒a*a+E⇒a*a+a. Identisches Blattwort; verschiedene Wurzelregel, also mehrdeutig.

Begründung: Nein. Ohne passende Einschränkung der Produktionsregeln erlaubt die Grammatik beide Bäume. Eine externe Rechenkonvention entfernt keine Ableitung. Eindeutigkeit muss die Grammatik selbst erzwingen.



1. A→aBB — Links muss genau eine Variable stehen. Die rechte Seite darf mehrere Terminale und Variablen enthalten.

2. Das abgeleitete Terminalwort (ε trägt kein Zeichen bei). — Der Baum beschreibt Struktur, die Blattfolge das Wort. Innere Variablen gehören nicht zum fertigen Wort.

3. Zwei verschiedene Bäume für dasselbe Wort. — Mehrdeutigkeit betrifft dieselbe Zeichenfolge mit verschiedener Struktur.

4. Nein, beide Reihenfolgen ergeben denselben Baum. — Der Baum hat immer S mit Kindern A und B und darunter a und b. Die Reihenfolge unabhängiger Ersetzungen ist irrelevant.

5. aabb — Jede Rekursion ergänzt außen ein a und b: S⇒aSb⇒aaSbb⇒aabb.

6. 8 — Drei Rekursionen liefern 6 Zeichen, der Abschluss ab weitere 2. Insgesamt 8.

7. E→E*E — Die äußerste Verknüpfung ist Multiplikation; die linke Teilstruktur verwendet Addition.

8. a+(a*a) — Ein T enthält keine Addition. Der rechte Summand darf dagegen ein Produkt aus T und F sein.

9. Nur diese Grammatik ist damit als mehrdeutig nachgewiesen. — Eine Sprache kann sowohl eindeutige als auch mehrdeutige Grammatiken besitzen. Inhärente Mehrdeutigkeit ist eine stärkere Aussage.

10. a+a*E — Die linkeste Variable ist das E direkt nach +; das rechte E bleibt stehen.

Chomsky-Normalform vollständig herstellen

Abrufwissen: A→BC oder A→a; nur ein nie rechts vorkommendes Startsymbol darf ggf. ε erzeugen. Nullable Variablen, ε-Elimination, Kettenelimination, Terminalersetzung, Binarisierung und Sprachkontrolle gehören zum vollständigen Verfahren.

Eigenauftrag

1. Neuer Start S0→S|ε; nullable S. Nach Entfernen von S→ε: S→aSbb|abb.

2. Kettenregel S0→S entfernen: S0→aSbb|abb|ε.

3. A→a, B→b. D→BB für das b-Paar und C→SD für den rekursiven Rest.

4. Ziel: S0→AC|AD|ε; S→AC|AD; C→SD; D→BB; A→a; B→b. Start S0, Variablen {S0,S,C,D,A,B}.

5. AC fügt ein a und zwei b hinzu, AD beendet mit abb; ε nur an S0. Daher genau $a^n b^{2n}$, n≥0. Alle Regeln in CNF, S0 niemals rechts.

Begründung: Nein. S0→S ist eine Kettenregel. Kopiere die Nicht-Kettenregeln von S zu S0 und entferne S0→S. S0→ε bleibt nur zur Erhaltung des leeren Wortes; S0 darf niemals rechts vorkommen.

1. A→BC — Zwei Variablen rechts sind erlaubt. aB ist gemischt, B allein eine Kettenregel.

2. Wenn S0 Start ist, nie rechts vorkommt und ε erhalten werden muss. — Die Ausnahme erhält das leere Wort an einem gesonderten Startsymbol.

3. {S, A, B, C} — Zuerst A,C; dann B über C; zuletzt S über AB. Terminal c trägt nichts zur ε-Ableitung bei.

4. {ABA, AB, BA, AA, A, B} — ABA, AB, BA, AA, A, B; das zusätzliche leere Ergebnis wird hier ausdrücklich nicht gefragt. Die zwei A-Vorkommen sind unabhängig.

5. A→a|b — A erreicht B und C per Ketten. Beide Terminalregeln müssen übernommen werden.

6. Beide →a|b — Die Kettenhülle beider Variablen ist {A,B}; beide können a und b erzeugen. Ein besuchtes Element wird nicht endlos verfolgt.

7. 2 — A→BX, X→CY, Y→DE. Zwei frische Variablen X,Y reichen.

8. A→TX — TX sind zwei Variablen. aX bleibt gemischt und TBC bleibt zu lang.

9. S0→S als Kettenregel entfernen und passende Regeln übernehmen. — Die ε-Ausnahme erlaubt keine Kettenregel. Übernimm die Nicht-Kettenregeln von S am neuen Start.

10. Dieselbe Sprache, aber nicht unbedingt dieselben Bäume. — Die Umformung ändert die Ableitungsstruktur, erhält aber alle Terminalwörter einschließlich eines gesondert behandelten ε.

11. Nein, längere Wörter könnten abweichen. — Endliche Worttests helfen Fehler zu finden. Ein vollständiger Beweis erklärt die Spracherhaltung jeder Umformungsphase.

Schritte ehrlich bewerten

1. Grundlagen · 1 Punkte

Drei Restzustände modulo 3 reichen. Ein a wechselt zyklisch, b bleibt im Zustand.

2. Grundlagen · 1 Punkte

Die zwei a des y-Blocks entfallen: $6-2=4$. Die sechs b bleiben unverändert.

3. Grundlagen · 1 Punkte

B direkt, A über B, S über AB. C erzeugt nur c.

4. Grundlagen · 1 Punkte

Genau zwei Variablen sind erlaubt. Die anderen Formen sind gemischt bzw. Kettenregeln.

5. Woche 3 · limits · 1 Punkte

L1 enthält acht Wörter und kann als endliche Vereinigung beschrieben werden. L2 ist nicht regulär: a^i und a^j, $i \neq j$, werden durch b^i unterschieden; unendlich viele Erinnerungsfälle wären nötig.

1 P.: L1 endlich und deshalb regulär; L2 nicht regulär mit unbeschränktem exaktem Vergleich bzw. unterscheidbaren Präfixen begründet.

6. Woche 3 · pumping · 2 Punkte

Angenommen regulär, Pumping-Länge $p \geq 1$. $w = 0^p 1^{3p} \in L$ und $|w| = 4p \geq p$. Für jede erlaubte Zerlegung liegt $y = 0^k$ im ersten Block, $1 \leq k \leq p$. $i=0$ entfernt k Nullen, lässt aber 3p Einsen. Die geforderte Beziehung ist verletzt (bei $p=k$ zudem $n \geq 1$).

1 P.: Regularität angenommen, beliebiges $p \geq 1$ und $w = 0^p 1^{3p}$ mit gültiger Mitgliedschaft/Länge.

1 P.: Jedes $y = 0^k$, $1 \leq k \leq p$; $i=0$ ergibt 3p Einsen statt $3(p-k)$, Widerspruch.

7. Woche 3 · syntax · 3 Punkte

Baum 1 hat E→E-E an der Wurzel und E→E*E rechts; Baum 2 hat E→E*E an der Wurzel und E→E-E links. Alle verbleibenden E→a. Beide Blattfolgen sind a-a*a, die Strukturen verschieden.

1 P.: Regelkonformer Baum für a-(a*a).

1 P.: Regelkonformer Baum für (a-a)*a.

1 P.: Identisches Blattwort a-a*a und verschiedene Wurzelregeln ausdrücklich begründet.

8. Woche 3 · cnf · 5 Punkte

Eine mögliche CNF: S0→TC|TB|b|ε; A→TC|TB; C→AB; T→a; B→b. Start S0; V={S0,A,C,T,B}. Der alte S wird nicht mehr gebraucht. TC fügt ein a/b-Paar hinzu, TB beendet mit ab, zusätzlich b und ε am Start. Andere sprachäquivalente CNF-Lösungen sind zulässig.

1 P.: Neues Startsymbol S0, ε erhalten, S0 nie rechts.

1 P.: Nullable A/S erkannt, A→ab ergänzt, alte ε- und Kettenregeln vollständig entfernt.

1 P.: Terminalvariablen und binäre Hilfsregeln korrekt.

1 P.: Vollständige Zielgrammatik enthält nur zulässige Formen.

1 P.: Sprache $\{a^n b^n : n \geq 0\} \cup \{b\}$ begründet, ε und b bleiben erhalten.

9. Woche 3 · cnf · 5 Punkte

Start S. S→AC|AD; C→SD; D→BB; A→a; B→b. V={S,C,D,A,B}. Alle Regeln sind CNF. AD erzeugt abb, AC erweitert um a links und bb rechts, also n≥1. Eine ε-Regel wäre falsch.

1 P.: ε ausgeschlossen, da keine ursprüngliche Ableitung ε erzeugt.

1 P.: Terminale durch A→a und B→b ausgelagert.

1 P.: Lange Regeln mit passenden binären Hilfsvariablen zerlegt.

1 P.: Vollständige Zielgrammatik und Start angegeben.

1 P.: Sprache $a^n b^{2n}$ mit n≥1 begründet; abb enthalten, ε ausgeschlossen.

Was du danach machst

1. Fehler nach Begriff, Notation, Verfahren, Rechenschritt, Begründung oder Zeit sortieren.

2. Die schwächste Aufgabe mit weniger Hilfe erneut aufbauen; danach neue Variante.

3. CNF-Regeln und ε-Ausnahme ohne Vorlage abrufen. In Woche 4 darauf aufbauend CYK und Kellerautomaten bearbeiten.

Quellen und Grenzen

Masterplan, PDF-S. 2–4 und 8; TGI_02_02_RegulaereSprachen, S. 70–81; TGI_03_KontextfreieSprachen, S. 7–33; Aufgaben (1), S. 9, 11, 15, 17; Aufgaben_Loesungen, S. 11, 13, 17, 19–21; Altklausur, S. 10. Alle Angaben sind PDF-Seiten. Eigene Erläuterungen und Übungsvarianten; Original-PDFs sind nicht eingebettet. Fehlerkorrekturen zu Aufgabe 11b und CNF-S. 33 sind in den Kapiteln erläutert. Keine Vorhersage von Aufgaben, Dauer oder Hilfsmitteln der echten Klausur.